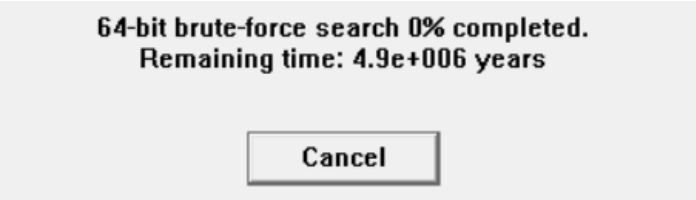
Álvaro Puebla Ruisánchez / Óscar López García

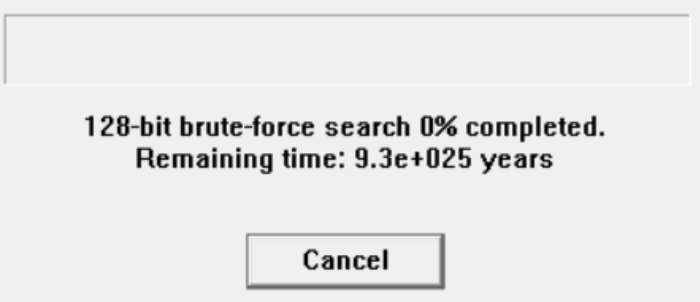
CYBERSECURITY REPORT – LABORATORY 5

1 Cryptanalysis for the symmetric algorithm:

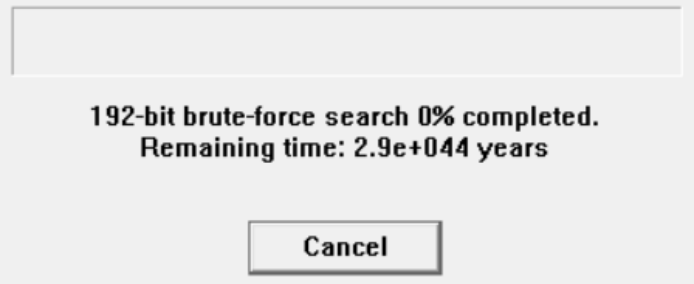
1.1 Evaluate the time needed to find a full key of 64, 128, 192, 256 bits:



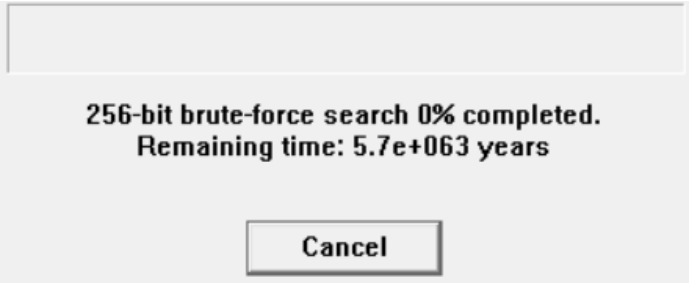
64 bits -> $4.9 * 10^6\ years$



128 bits -> $9.3 * 10^{25}\ years$



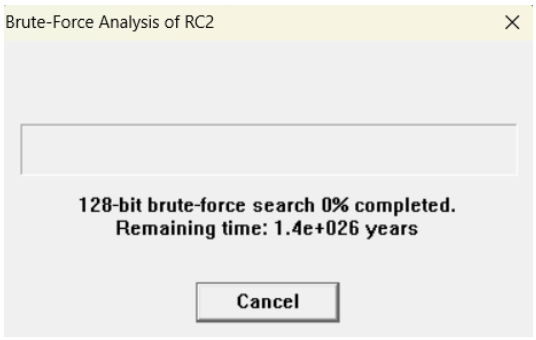
192 bits -> $2.9 * 10^{44}\ years$

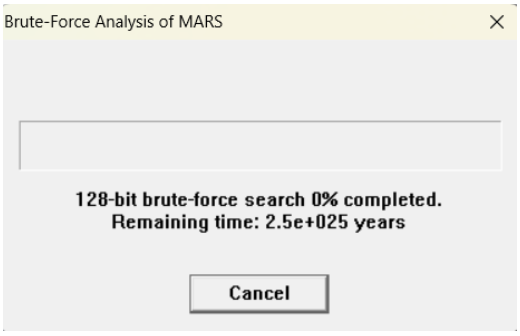


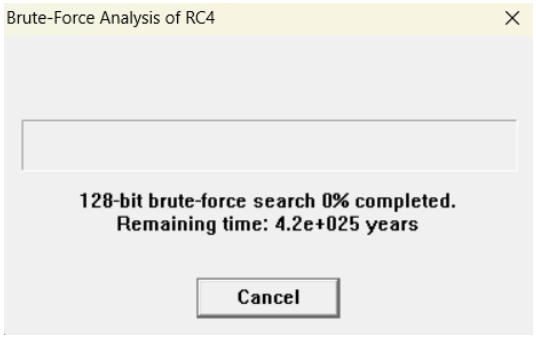
256 bits -> $5.7 * 10^{63}\ years$

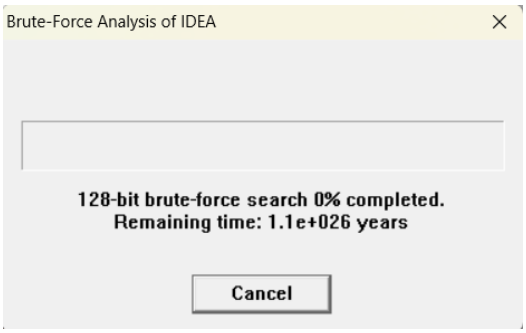
The **time grows exponentially as the key increases**.

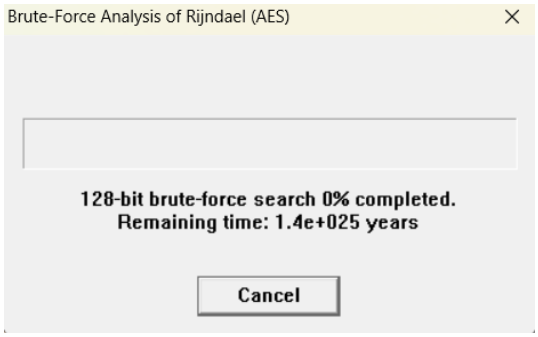
1.2 Compare the time it takes to find keys of the same length (e.g. 128 bits) for different algorithms:











Nearly all have the same range, **about 10^{26} years**, a lot for my small computer, a faster one, will reduce the time to finish.

So: **for pure brute force, same length -> same order of time.**

1.3 Determine the time needed to find a key if the unknown number of bits of the key is: 4, 8, 12, 16, 20, 24, 30, 34. (One star in the key equals 4 bits):

- 4 -> <1s
- 8 -> <1s
- 12 -> <1s
- 16 -> 1s
- 20 -> 7s
- 24 -> 2 min 1s
- 28 -> 32 min
- 34 -> 8 h 44 min

The time grows **exponentially** with unknown bit count; measured runtimes vary with machine/implementation.

1.4 Check whether the position of the unknown bits in the key affects the key search time:

No (for blind brute force). If you brute-force every combination of the unknown bits, position doesn't change count of trials. Caveat: if you exploit structure (known plaintext, easy checks) or use cryptanalytic shortcuts, placement might matter.

1.5 Evaluate the performance of the cryptanalysis tools:

Strengths: great for teaching, can automate known-plaintext / brute-force experiments, shows exponential scaling clearly.

Weaknesses: limited by host CPU/GPU speed; results are machine-dependent; some tools don't simulate optimized hardware (ASIC/FPGAs).

Practical: very effective for small unknown keyspace (≤28–34 bits). Not useful for full modern keys (≥128 bits).

1.6 Do we always get the correct key?

Yes if you exhaust the key space or have sufficient distinguishing data. Practically: Yes for small spaces; for very large spaces you *can* still be correct in theory but infeasible in time.

1.7 Does the number of the bits search affect the quality of the key being restored?

No. It affects **time** and feasibility, not the correctness of the recovered key (if search is exhaustive).

1.8 Does the position of unknown bits affect the quality of the key being:

No, it does not matter if you put them at the start, in the middle or at the end, it is the same.

1.9 Can modern block algorithms be considered safe (referring to the above experiments)?

Yes, modern block algorithms (AES, 3DES, etc.) can
be considered safe when used with appropriate key lengths. The experiments demonstrate that:

With keys of 128 bits or more, the time required for a brute force attack is too high to get it

Brute force attacks are impractical for modern algorithms with full-length keys

Security depends on using the complete key length without known bits

1.10 What length of the key offers us a sufficient level of security for particular algorithms? Why?

128 bits is sufficient security for most applications because
even checking 1 trillion keys per second, it would take 10^22 years to try all combinations.

1.11 Does the size of the ciphertext influence the possibility of breaking it?

No, it doesn't affect directly. But as bigger is the ciphertext, the most probable is to detect patterns in the algorithms that reflect them in the ciphertext.

1.12 Does the format and earlier processing of the document affect the possibility of its cryptanalysis (e.g. compression etc.)?

Yes. The way a document is formatted or processed before encryption can affect how easy it is to break. For example, compression removes patterns, which can make cryptanalysis harder. On the other hand, predictable formatting can create patterns that make attacks easier.

1.13 How many possible passwords can we check in a year of continuous work on one computer, which checks one million passwords per second (2^{20})?

$$\frac{Checks}{year} = 1\,000\,000 * 365.25 * 86400 = \ 31\,557\,600\ checks\ per\ year$$

1.14 What does this result say about the security of modern algorithms?

It shows modern key lengths (≥128 bits) are **practically secure** against brute force on current/near-future hardware. Security also depends on correct use (no weak keys, no side-channel leaks, good randomness).

2 Cryptanalysis for the asymmetric algorithm:

2.1 Please try to factorize the number created by the combination of

- Student's ID number
- Year
- Month
- Day
- Hour
- Minutes

Example: 123456202011051408

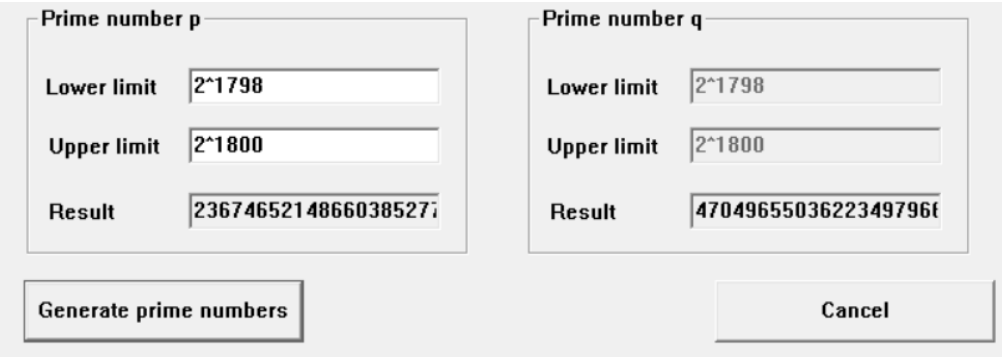
Tab: Indiv. Procedures -> RSA Cryptosystem -> Factorization of a number

The number is: $293867202511061655 = 3 * 5 * 7 * 19 * 147301855895269$ in 0.044 seconds

2.2 Please check how the time of searching for prime numbers increases with the value of searching interval:

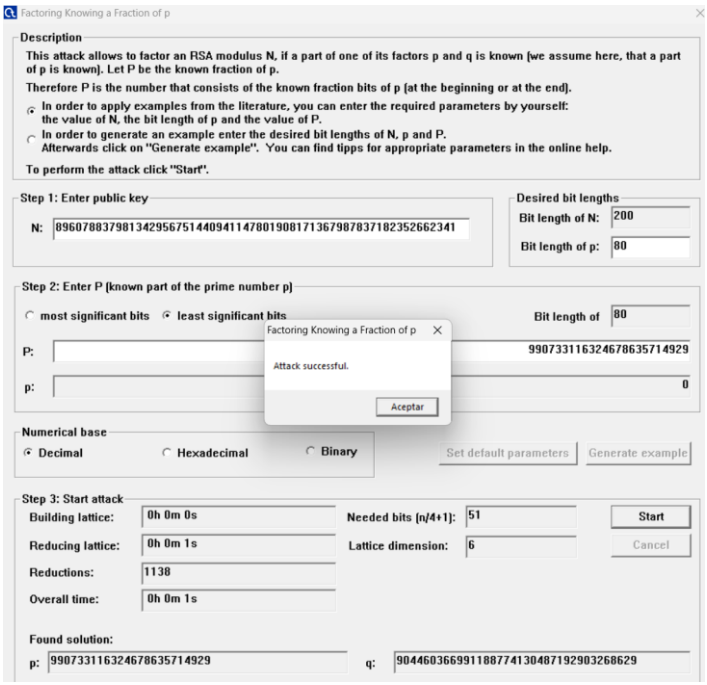
Tab: Indiv. Procedures -> RSA Cryptosystem -> Generate Prime Numbers

Time grows roughly **exponentially** with the interval, doubling the interval about squares the time. Prime density ($\approx 1/\ln x$) and per-test cost (polynomial in bit-length) change constants, but not the linear dependence on interval length.

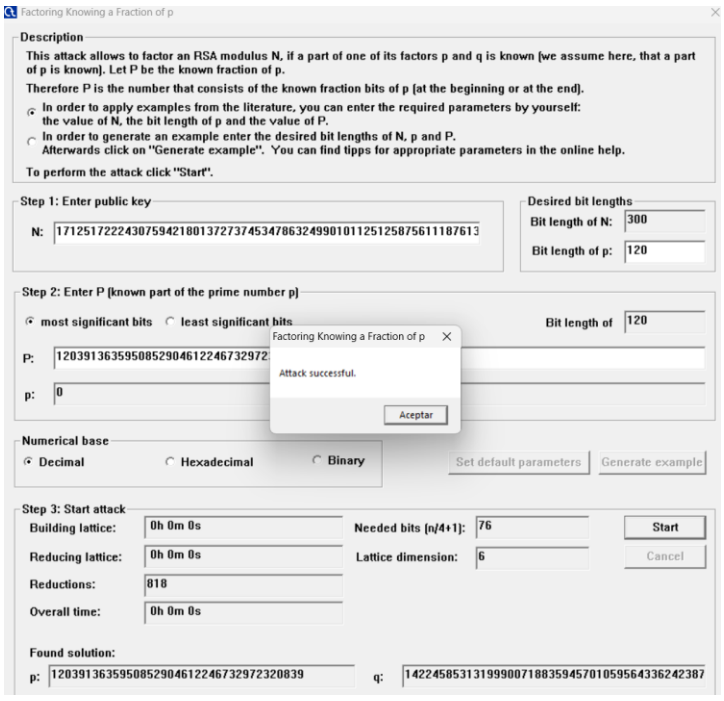


2.3 Check how different parameter values influence the effectiveness and time of need for the RSA module N factorization attack: Length N, Length p, Length of known bit string P (lab_5_numbers.txt) Tab: Analysis -> Asymmetric encryption -> Lattice-Based Attacks on RSA -> Factoring with a hint

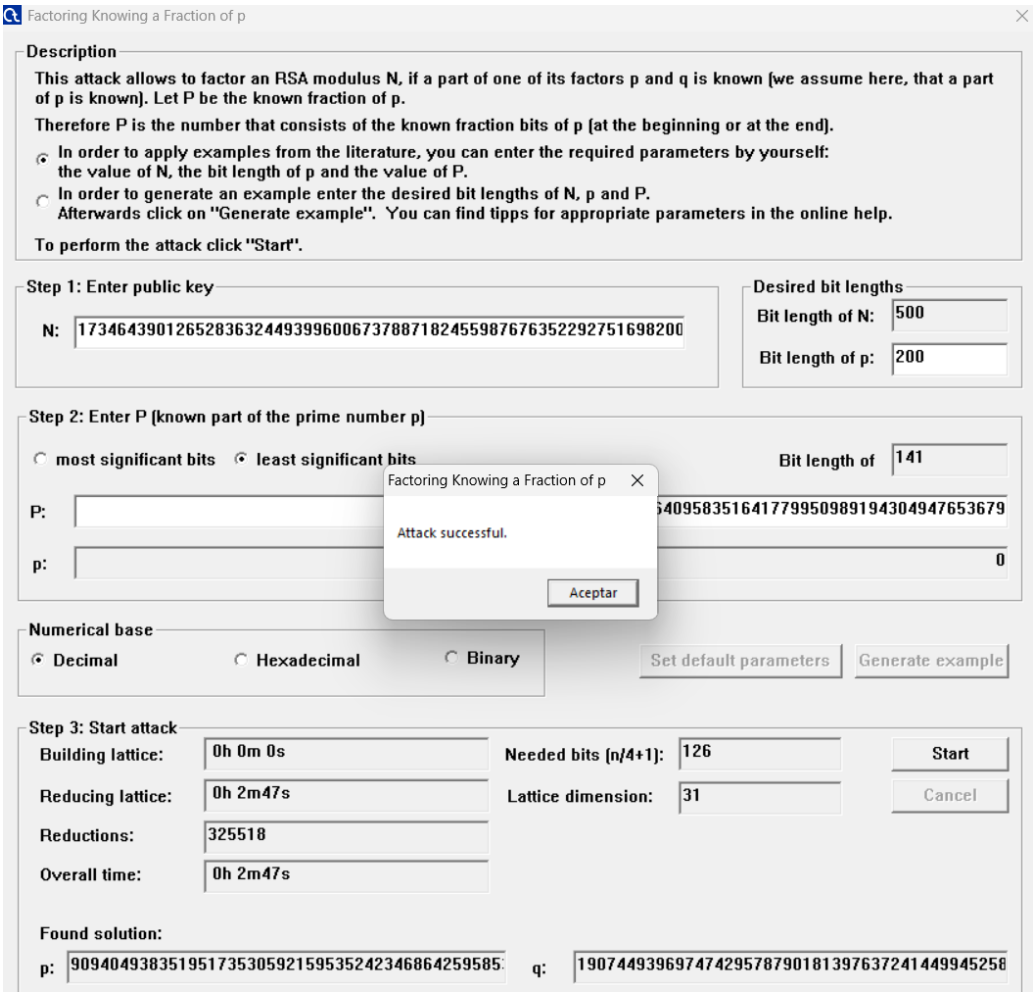
Example 1:



Example 2:



Example 3:



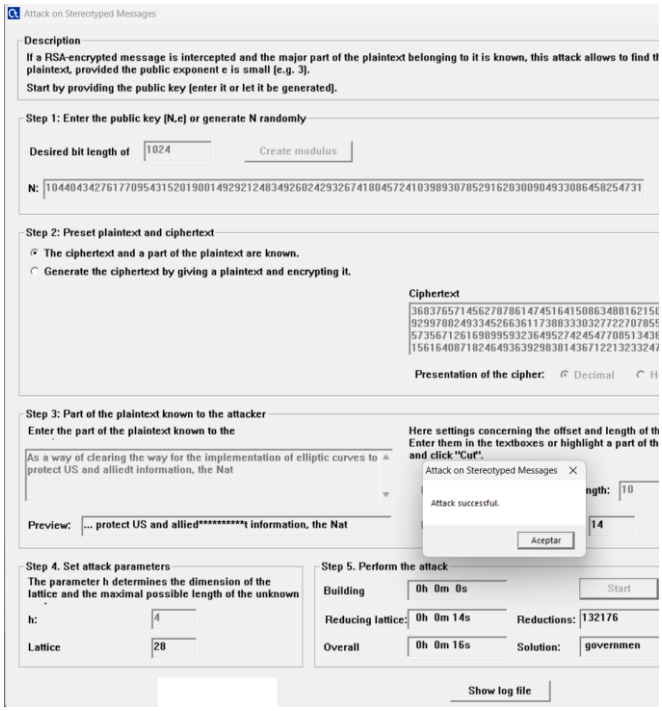
The **more consecutive bits of one prime p you know, the search space shrinks exponentially**. **Knowing k bits reduces the brute-force cost by about 2^k**, so recovery time falls sharply and beyond small thresholds you can recover p efficiently (or use Coppersmith lattice methods).

If you recover p **then $q = \frac{N}{p}$** is trivial; if you don't know enough bits, brute force is infeasible for real RSA and you must resort to lattice attacks (which have their own bit-length thresholds).

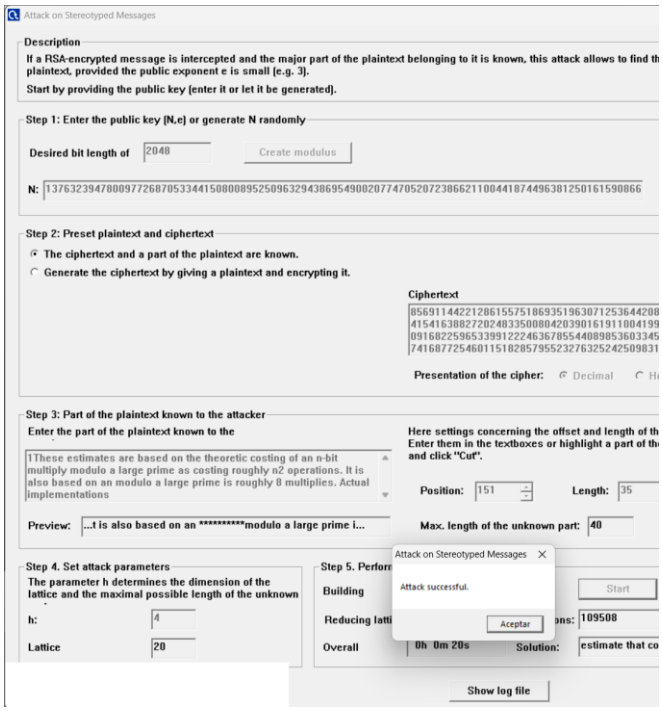
2.4 Find the missing text fragment from lab_5_number.txt using the attack method available in the tab:

Tab: Analysis -> Asymmetric encryption -> Lattice-Based Attacks on RSA -> Attack on Stereotyped Messages

Example 1: Solution: “government”



Example 2: Solution: “estimate that computing an inverse”

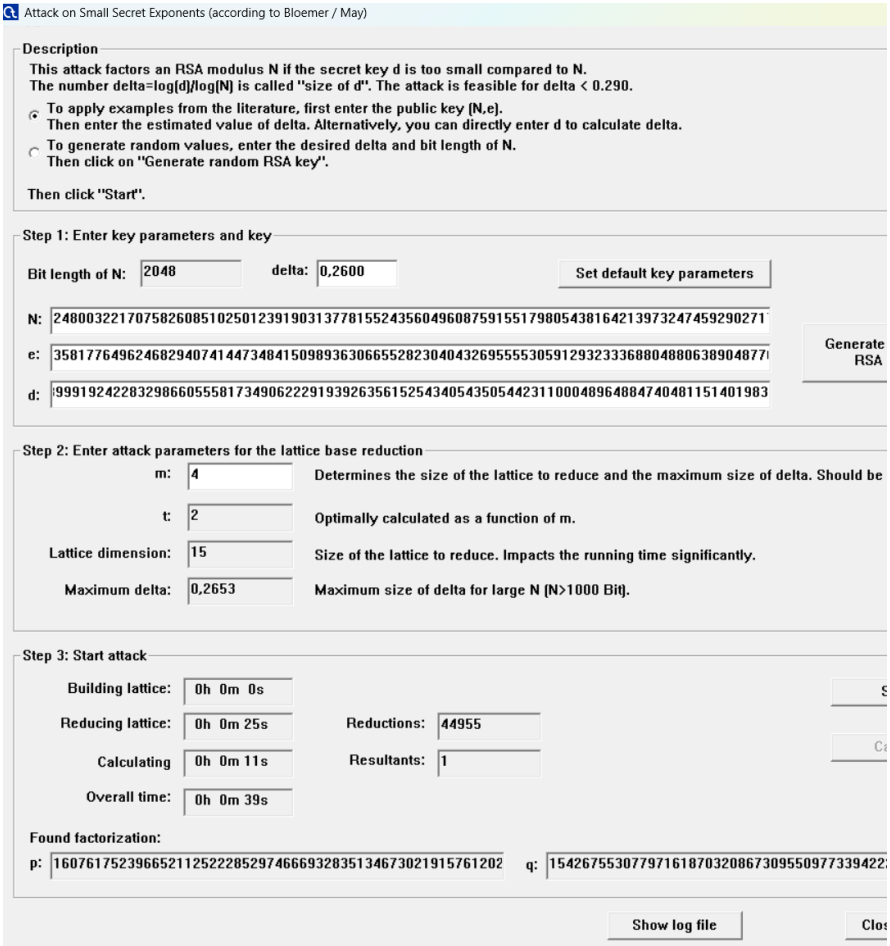


2.5 Factorize the modules from the file lab_5_number.txt using the additional data contained in this file (values: d, e, delta, ...) and the attack method available under the tab:
Tab: Analysis -> Asymmetric encryption -> Lattice-Based Attacks on RSA -> Attack on Small Secret Keys

Example 1:

Example 2:

Example 3:



2.6 What is the minimum length of the module (number N) of the RSA algorithm, which guarantees that its distribution (finding its prime factors) will be difficult enough?

Recommendation: use **≥2048-bit RSA** (prefer **3072-bit** for stronger margin).

Why: 2048 ≈ 112-bit classical security; 3072 ≈ 128-bit; 4096 gives extra margin.

Crack time:

- **1024-bit: feasible for well-resourced attackers** -> months to years.
- **2048-bit:** requires **millions–billions CPU-core years** (decades–centuries on typical clusters) -> practically infeasible today.
- **3072/4096-bit:** orders of magnitude harder than 2048.

2.7 Is it possible to carry out an effective factoring attack based on partial knowledge of the value of one parameter for the module length assumed in the previous point as safe? (task 3)?

Yes, possible, but only in specific cases. Partial knowledge can let attackers break a 2048-bit modulus when the **leaked** or chosen value **is large enough or the secret is unusually small**.

Main risky situations:

- private exponent d is *too small* (Wiener, Boneh–Durfee ranges);
- many contiguous bits of d, d_p or d_q are leaked;
- many bits of a prime factor p or q are known;
- keys share primes or are generated with poor/randomness.

Practical point: single-bit leaks or tiny accidental leaks usually don't break a properly generated 2048-bit key; large, **structured leaks or deliberately small d do.**

Mitigations: generate full-size random keys with standard libraries, avoid small private exponents, protect private values (HSMs, side-channel defenses), and use secure padding (OAEP) or hybrid encryption.

2.8 In what cases can RSA encryption be threatened by the attack in point 4?

RSA encryption can be threatened by **stereotyped-message attacks** when message structure is predictable.

1. The danger appears mainly with **small public exponents** (e.g. 3, 5, 7).
2. If most of the plaintext is known and **only a small part is secret**, attackers can model it as a small-root problem.
3. **Lattice-based (Coppersmith/Håstad)** methods can then recover the hidden fragment without factoring N.
4. The risk grows if **no randomized padding (textbook RSA)** is used.
5. Sending the **same plaintext to several recipients with the same small e also enables recovery**.
6. **Mitigation:** always use **RSA-OAEP** padding or hybrid encryption to prevent this.

2.9 In what cases can RSA encryption be threatened by the attack in point 5?

RSA is threatened by small-secret-key attacks whenever the private secret is **too small or partially exposed**, so an attacker can turn the public data N, e (and any leaked bits) into a Diophantine/lattice problem that yields d or the factors.

Concrete cases:

1. **Private exponent d is small** (Wiener: $d < N^{\frac{1}{4}}$ and Boneh–Durfee and lattice variants go up to about $d < N^{0.292}$ with strong tools).
2. **Some bits of d (or CRT secrets d_p, d_q) leak** via side-channels, timing, or poor handling, partial leakage greatly reduces attack cost.
3. **Poor key generation / deliberate small d** for speed (choosing a small private exponent) or reusing parameters that make d small relative to N.
4. **Correlated/related keys or shared primes** (e.g. same p across keys, structured RNG) that let attackers set up equations solved by lattices.
5. **Implementation leaks** (timing, fault attacks) that expose enough secret information to make lattice/continued-fraction attacks feasible.

Mitigation: generate keys with standard libraries (full-size random d), avoid intentional small d, protect CRT values and private bits from leakage, and use recommended key sizes and secure padding.